

```

        raise TypeError('Second object was not a Rectangle')
    if self.area == oOtherRectangle.area:
        return True
    else:
        return False

# Magic method called when you compare
# two Rectangle objects with the < operator
def __lt__(self, oOtherRectangle): ❷
    if not isinstance(oOtherRectangle, Rectangle):
        raise TypeError('Second object was not a Rectangle')
    if self.area < oOtherRectangle.area:
        return True
    else:
        return False

# Magic method called when you compare
# two Rectangle objects with the > operator
def __gt__(self, oOtherRectangle): ❸
    if not isinstance(oOtherRectangle, Rectangle):
        raise TypeError('Second object was not a Rectangle')
    if self.area > oOtherRectangle.area:
        return True
    else:
        return False

def getArea(self):
    return self.area

def draw(self):
    pygame.draw.rect(self.window, self.color, (self.x, self.y, self.width,
self.height))

```

Listing 9-6: The `Rectangle` class

The methods `__eq__()` ❶, `__lt__()` ❷, and `__gt__()` ❸ allow client code to use standard comparison operators between `Rectangle` objects. To compare two rectangles for equality, you would write:

```

if oRectangle1 == oRectangle2:

```

When this line runs, the `__eq__()` method of the first object is called, and the second object is passed in as the second parameter. The function returns